

GRAND CHALLENGE MONOGRAPH COLLECTION

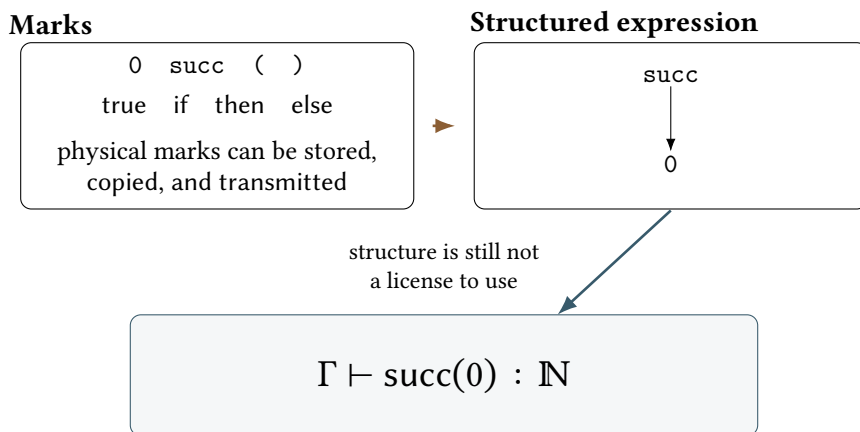
TYPE THEORY

Volume I: Judgment

Canonical Plate Folio I

Finished Plates 1–42

Before execution



The added layer: a formal system licenses a claim about the expression.
Parsing organizes marks; typing situates the expression in a discipline of admissible use.

Figure 1: **Plate 1. Before execution.** Symbol manipulation, parsing, and typing are distinct layers. The diagram does not claim that every meaningful computation must be typed; it isolates the additional structure a typed formal system contributes.

Anatomy of a judgment

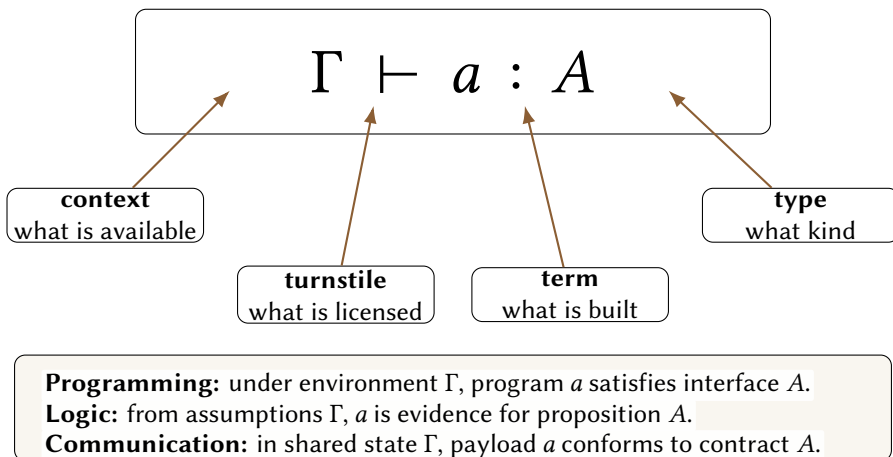
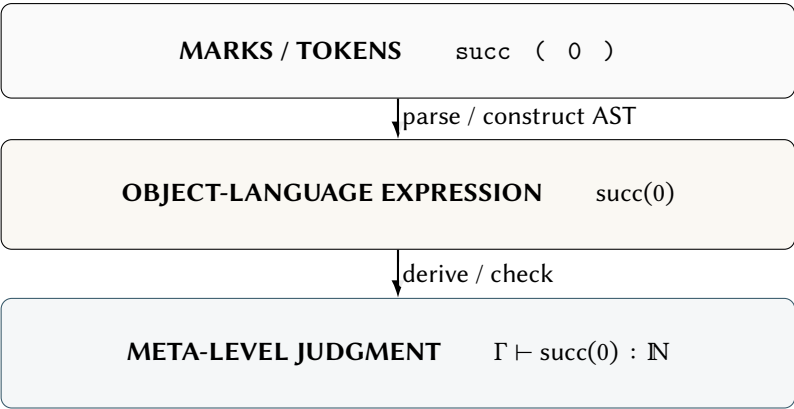


Figure 2: **Plate 2. Anatomy of a judgment.** The four visible parts of $\Gamma \vdash a : A$ answer different questions. The three readings at the bottom are interpretations of one formal shape, not assertions that programming, logic, and protocol semantics are identical in every respect.

Marks, expressions, judgments

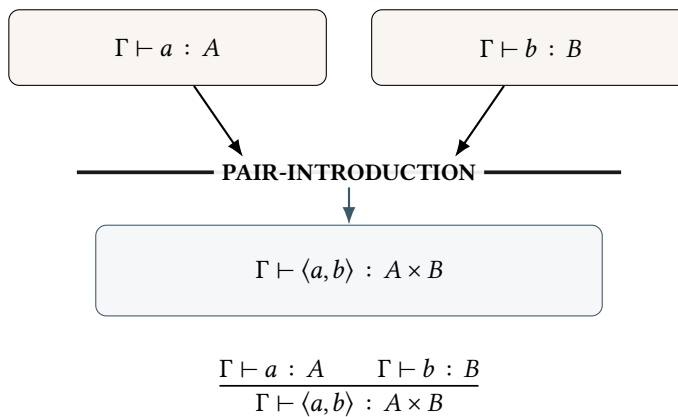


Each downward move adds commitments.

A term is not itself a claim that the term is well formed, typable, equal, or true.

Figure 3: **Plate 3. Three levels that must not be collapsed.** The stack is conceptual, not an implementation mandate: a compiler may interleave parsing and checking, but the distinctions between concrete marks, object-language syntax, and meta-level assertions remain useful.

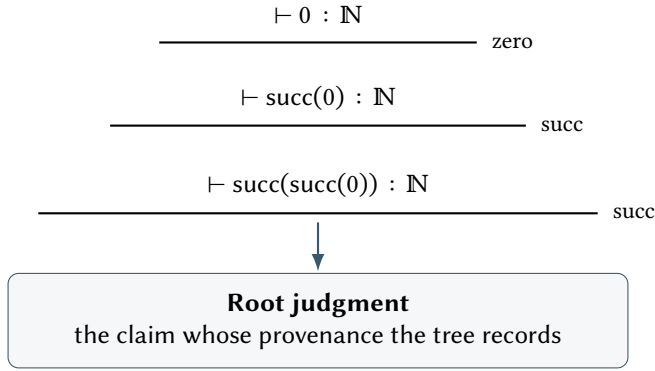
A rule manufactures a licensed judgment



The rule bar is not causal machinery. It records a license:
if the premises are derivable, so is the conclusion.

Figure 4: **Plate 4. Inference as licensed construction.** The press-like layout is mnemonic only. An inference rule need not describe temporal execution; it specifies a valid step in a derivation.

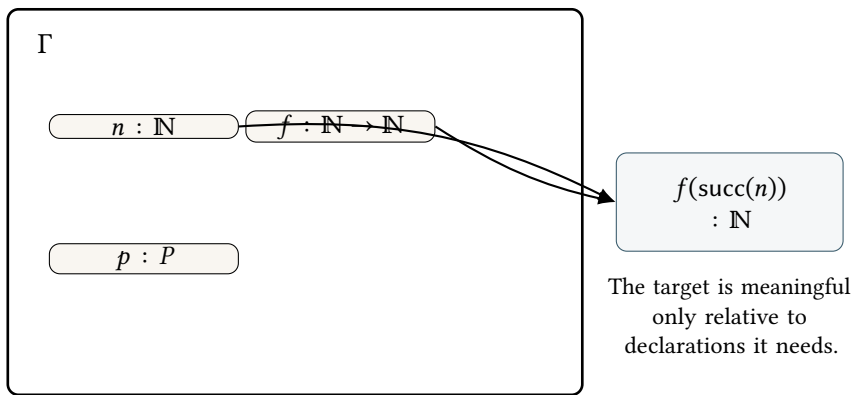
Derivation tree for two



Every edge can be audited against a rule.
A correct-looking conclusion with a forged premise is not a derivation.

Figure 5: **Plate 5. Derivation as provenance.** The tree records why a judgment is licensed. Its vertical orientation follows proof-theory convention rather than the chronological order in which an implementation must construct the tree.

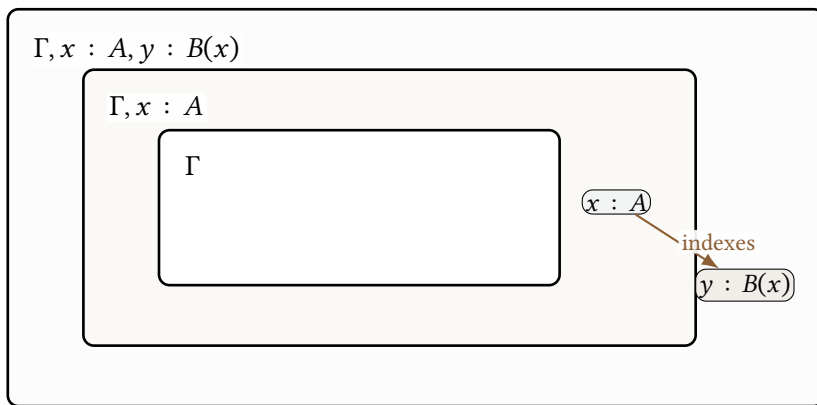
Context as an informational room



Available does not mean used.
Weakening permits $p : P$ to remain unused.

Figure 6: **Plate 6. A context as available information.** The room metaphor emphasizes locality and availability. Contexts are formal sequences of declarations, not literal containers, and richer type theories impose dependency-sensitive ordering.

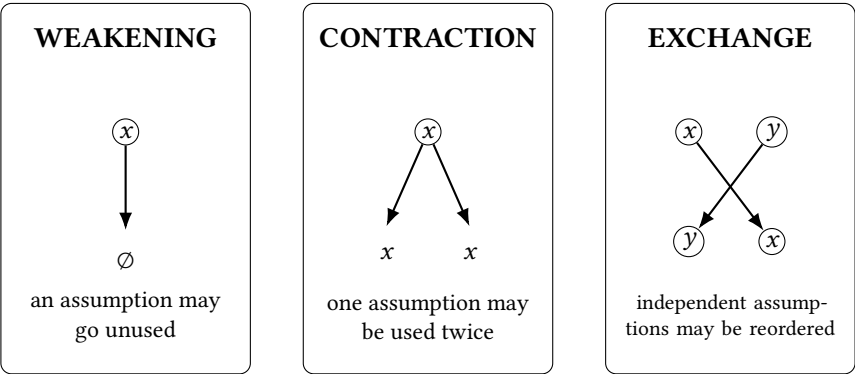
Context extension is world extension



Later declarations may be meaningful only because earlier information is already available.

Figure 7: **Plate 7. Context extension.** The nested frames depict inclusion of available assumptions. They should not be read as set inclusion in the dependent case: order and dependency are part of the formal structure.

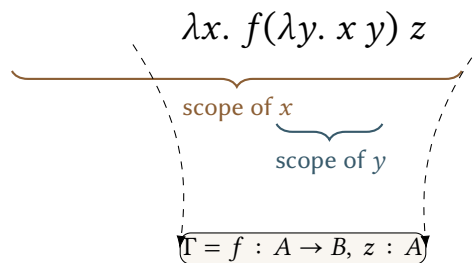
Structural rules are resource policies



Linear and relevance-sensitive systems deliberately revoke some of these privileges.

Figure 8: **Plate 8. Structural rules as computational assumptions.** Disposal, duplication, and reordering are represented as resource operations. The picture does not imply physical copying costs; it identifies permissions encoded by ordinary structural typing.

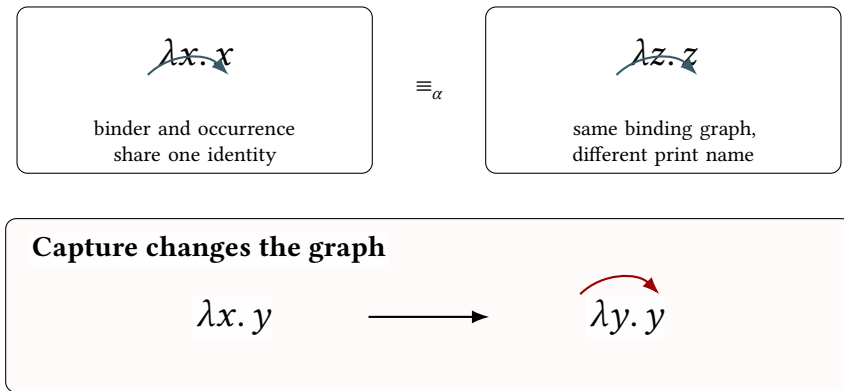
Bound and free variables



x and y are licensed by binders; f and z must be licensed by the surrounding context.

Figure 9: **Plate 9. Scope is an information boundary.** Braces and tethers show which occurrences are locally bound and which remain obligations on the external context. Printed position is incidental; binding structure is the invariant.

Alpha-equivalence preserves binding identity



Renaming is valid only when it leaves the binder-occurrence relation unchanged.

Figure 10: **Plate 10. Alpha-equivalence.** Variable names are replaceable labels on binding identities. The lower panel shows why freshness conditions matter: a textual rename can turn an external dependency into a local one.

A type family assigns a space of possibilities

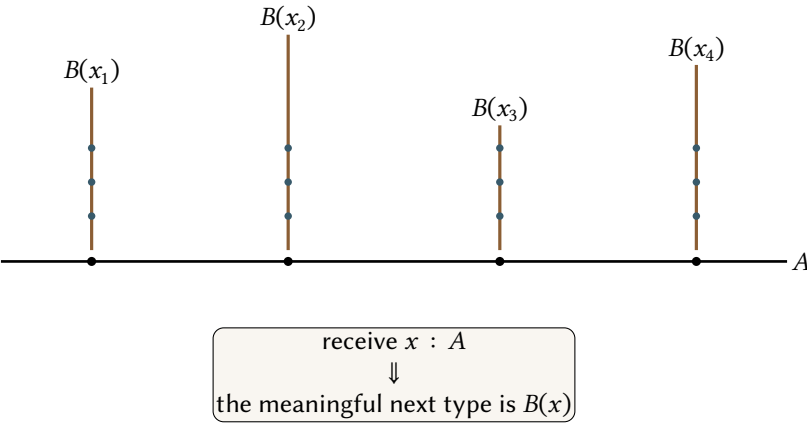
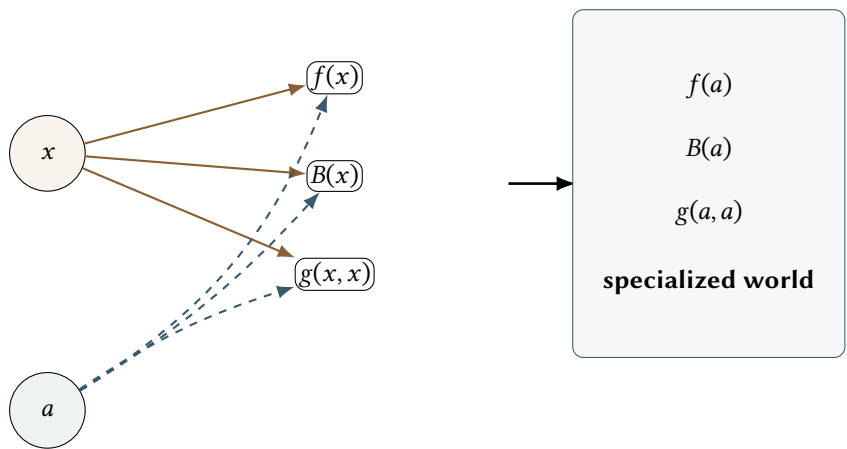


Figure 11: **Plate 11. Fibres of a type family.** Geometry supplies an intuition: a base point chooses a corresponding fibre. At this stage the picture is mnemonic; Volume II will make dependent formation and substitution precise, and later semantic volumes will say when spatial language is literal.

Substitution is information in motion



Supplying $a : A$ discharges the hypothetical source $x : A$ and specializes every genuine dependency.

Figure 12: **Plate 12. Substitution as dependency rewiring.** The graph picture tracks dependency, not machine memory addresses. Capture avoidance ensures that replacing one dependency does not silently alter the binding identity of another.

Capture avoidance preserves external dependencies

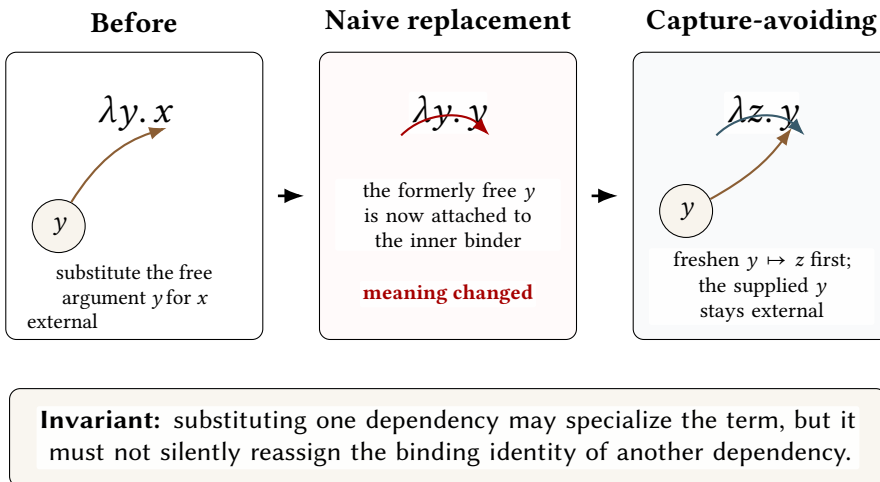


Figure 13: **Plate 13. Capture-avoiding substitution.** The tether picture records binding identity, not pointer structure in an implementation. Freshening a binder is required exactly when a free variable in the incoming term would otherwise become captured.

Beta reduction closes a hypothetical world

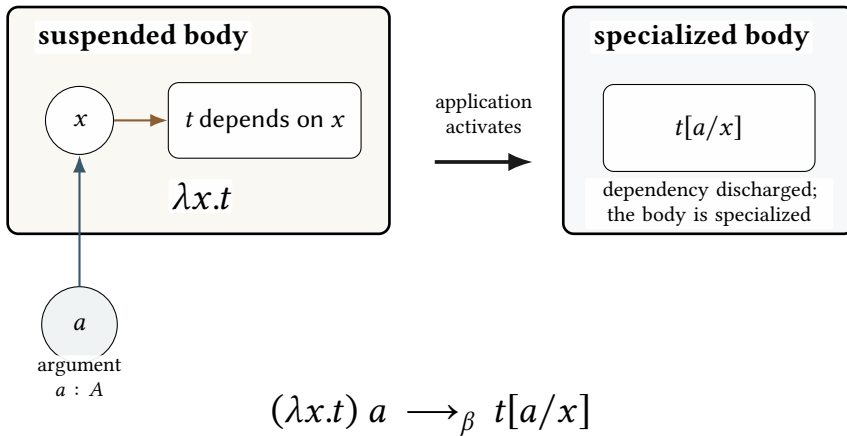
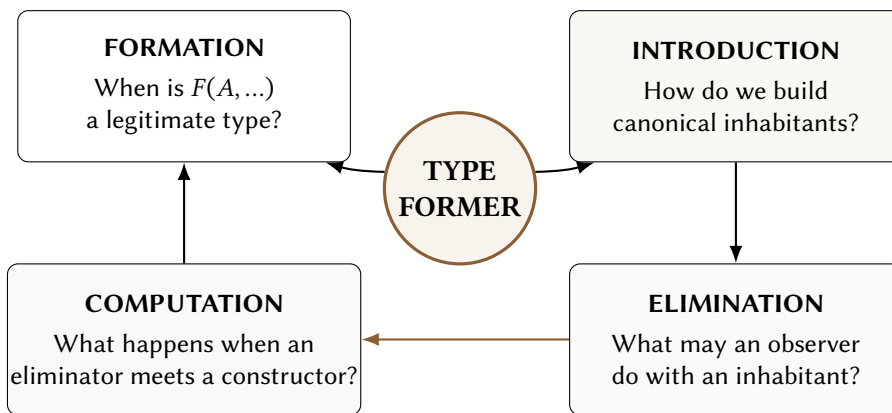


Figure 14: **Plate 14. Beta reduction.** The socket metaphor emphasizes discharge of a hypothetical dependency. It does not prescribe an evaluator's runtime representation: closures, environments, graph reduction, and substitution machines can realize the same rule differently.

The four questions for every type former



A new connective is not specified by syntax alone. These four questions determine its admissible objects, its evidence, its observations, and the reductions generated where evidence meets observation.

Figure 15: **Plate 15. The type-former compass.** Formation, introduction, elimination, and computation are a design discipline rather than four universally independent rule schemas. Some systems package or derive parts of the quartet differently, but the four questions remain a useful audit.

Harmony: observation recovers construction

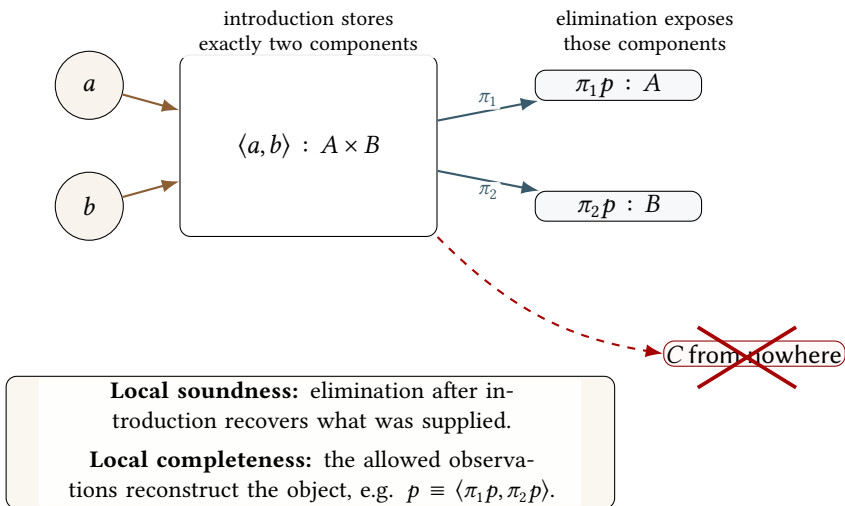


Figure 16: **Plate 16. Proof-theoretic harmony.** The product container is mnemonic. Harmony relates introduction and elimination rules; not every type former has literal fields or projections.

A function is a staged capability

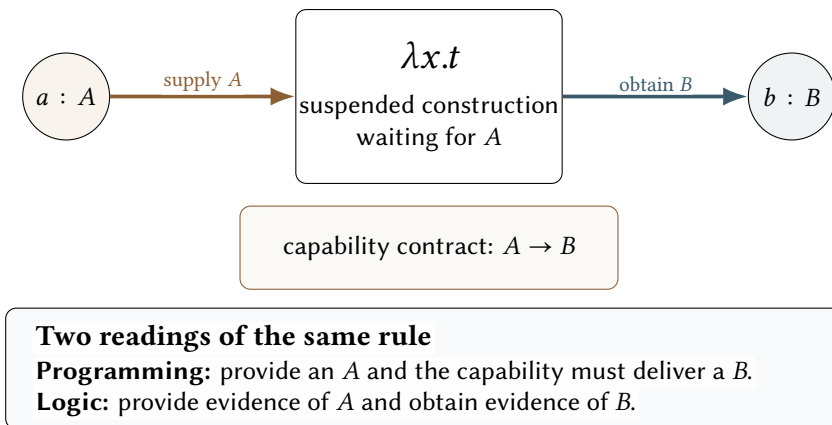
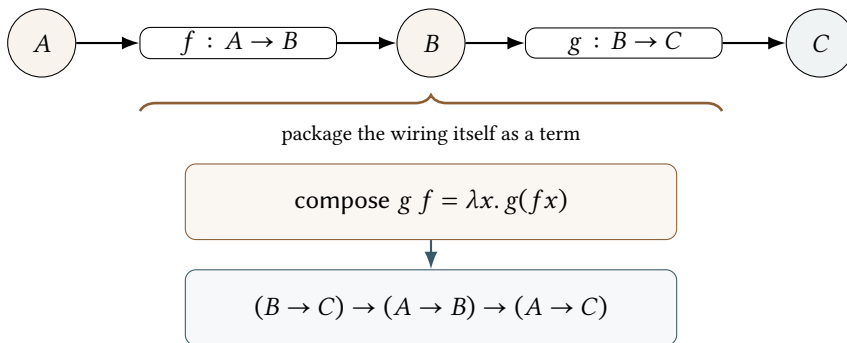


Figure 17: **Plate 17. Function as staged capability.** The diagram emphasizes the behavioral contract of $A \rightarrow B$. It does not imply eagerness, a particular calling convention, or that all computations are best modeled as total functions.

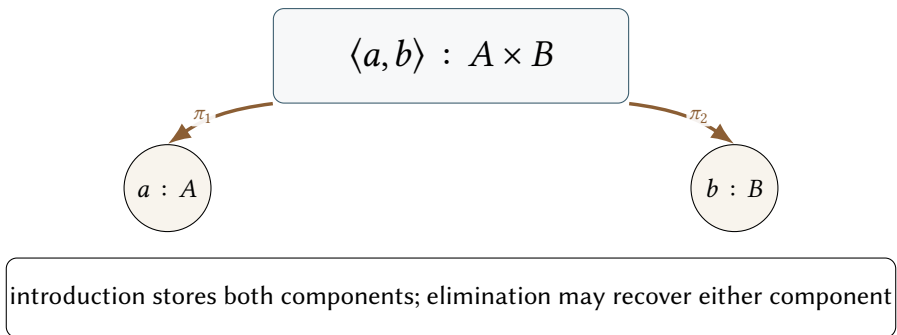
Higher-order composition: capabilities become data



The striking step is not composition itself; it is that f and g are ordinary inhabitants that may be accepted, returned, stored, and composed by another program.

Figure 18: **Plate 18. Higher-order composition.** The wiring diagram depicts extensional flow from A through B to C . It suppresses evaluation strategy and internal effects; two functions with the same arrow types need not have identical operational behavior.

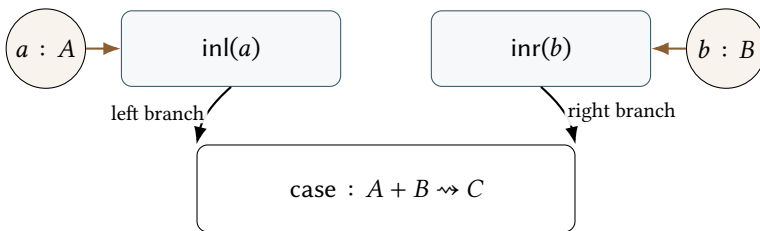
Product: both pieces are available



A product is stronger than “two possible values.” It promises simultaneous possession: every inhabitant supports both observations.

Figure 19: **Plate 19. Product.** The two projection arrows expose the observations guaranteed by $A \times B$. The picture suppresses sharing and evaluation cost: projecting a component need not imply that both components were eagerly computed.

Sum: one payload, with its branch tag

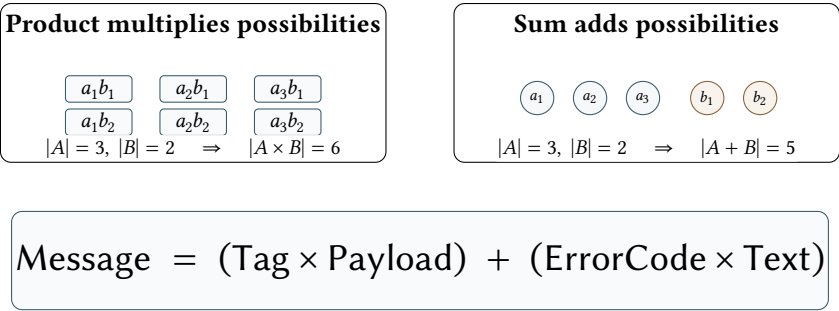


elimination is exhaustive because the type lists the canonical alternatives

The tag is computational evidence. A consumer may not discard the distinction until it has supplied behavior for both constructors.

Figure 20: **Plate 20. Sum.** A sum contains exactly one branch's payload together with a constructor tag. The diagram shows the canonical two-constructor case; larger variants are repeated sums or primitive variants with more constructors.

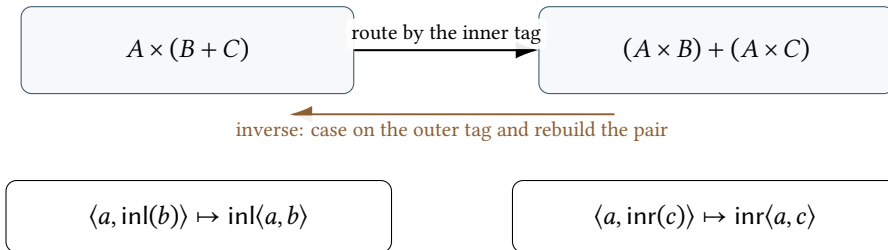
The algebra of data



Sums choose constructors; products collect fields. Nested together, they form the grammar of ordinary algebraic data types. For finite types the arithmetic is literal cardinality arithmetic.

Figure 21: **Plate 21. Algebra of data.** Products and sums behave like multiplication and addition of finite cardinalities, and the same constructors organize richer data shapes. The arithmetic analogy is exact for finite sets, but cardinality alone forgets typing, labels, recursion, and computational behavior.

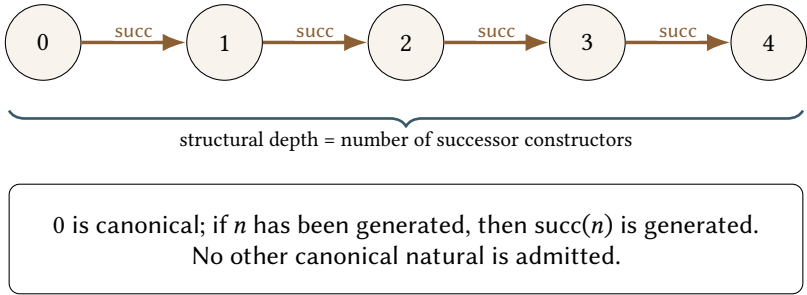
Distributivity is executable rewiring



The isomorphism is not a symbolic identity imposed from outside. It is witnessed by programs whose composites normalize to the corresponding canonical inputs.

Figure 22: **Plate 22. Distributivity as program.** The two sides contain the same information organized differently. The arrows denote explicit total functions, not definitional equality: an implementation may store or evaluate the two representations differently.

Natural numbers are generated, layer by layer



The eliminator mirrors this generation rule: one case for 0, one step for succ.
Generation and elimination are two faces of the same inductive structure.

Figure 23: **Plate 23. Natural numbers as generated structure.** The ladder depicts constructor depth rather than numerical distance in a metric space. The ellipsis beyond the shown numerals is implicit: the inductive rule permits arbitrarily many finite successor layers.

The recursor follows construction history

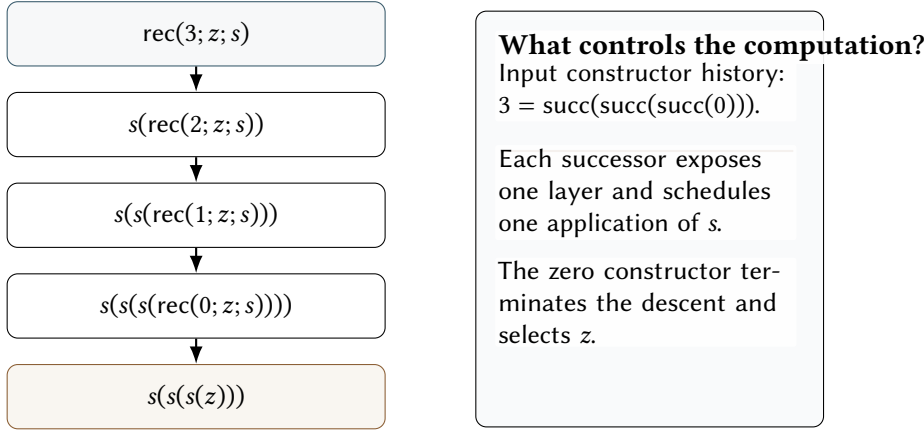


Figure 24: **Plate 24. Recursor follows construction history.** The reduction trace is for the numeral 3 and a unary step $s : C \rightarrow C$. It illustrates structural recursion, not every possible evaluation strategy or optimized implementation of primitive recursion.

Induction transports evidence through a family

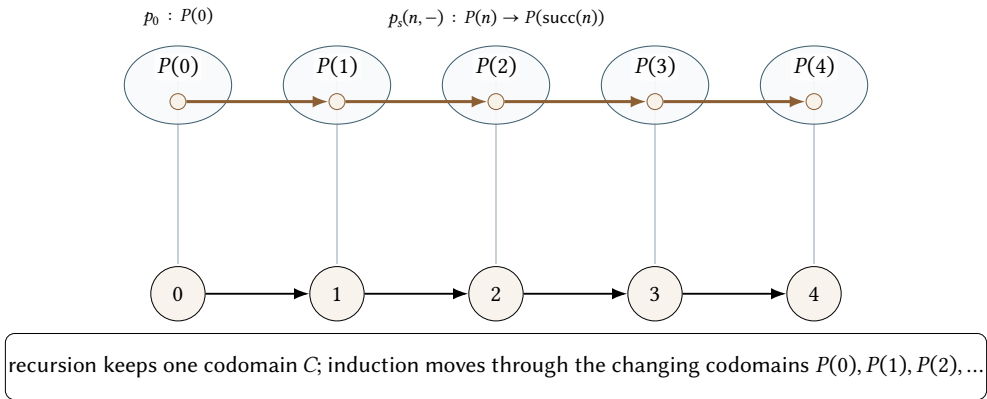


Figure 25: **Plate 25. Induction fibers.** The vertical fibres depict a type family $P(n)$ indexed by naturals, and the warm points and arrows depict evidence transported by the induction step between adjacent fibres. The geometry is schematic: no metric or literal spatial embedding of types is assumed.

One word, several equality judgments

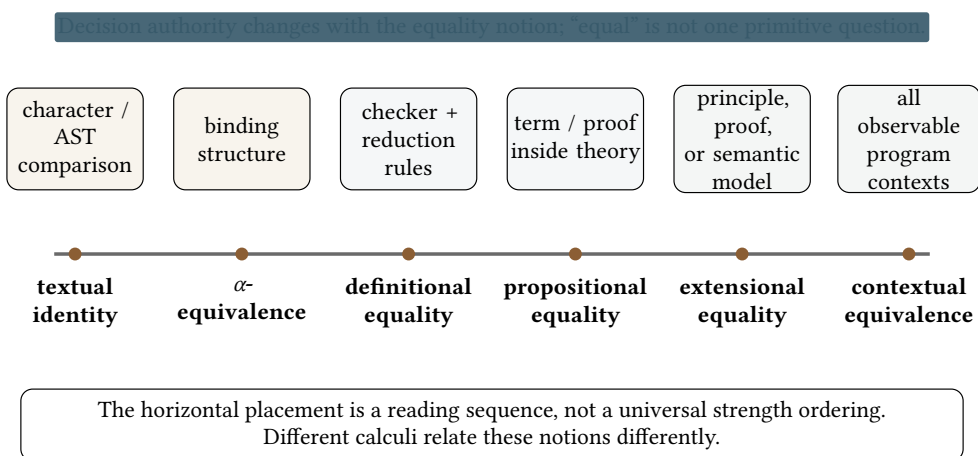


Figure 26: **Plate 26. Equality spectrum.** The same phrase—“these programs are equal”—can invoke syntax, binding structure, the checker’s conversion algorithm, internal proof, an extensional principle, or contextual observation. The sequence is deliberately not drawn as implication arrows: there is no theory-independent total ordering of these notions.

Confluence is a claim about an actual reduction graph

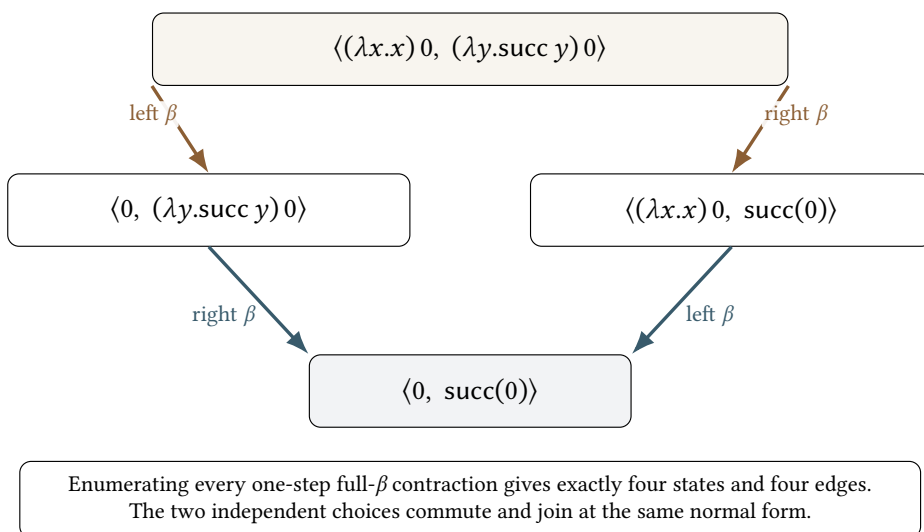


Figure 27: **Plate 27. A computed confluence diamond.** This is the complete full- β reduction graph of the displayed product term, not a generic diamond icon. Reducing either independent redex first produces a distinct intermediate term, and the remaining contraction joins both paths at $\langle 0, \text{succ}(0) \rangle$. The machine-readable edge list is shipped with the laboratory sources.

Equality has an automation boundary

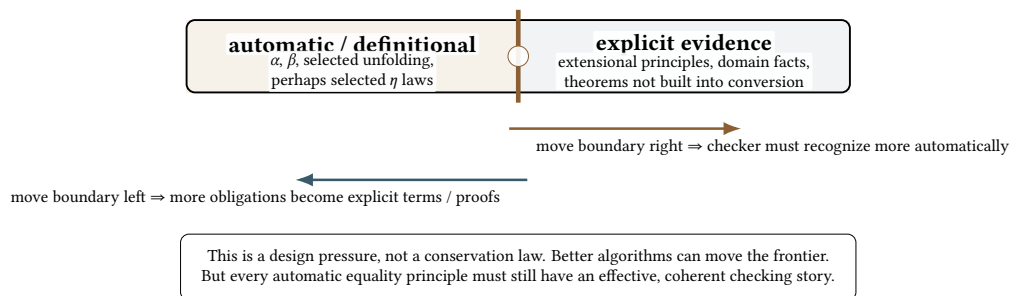
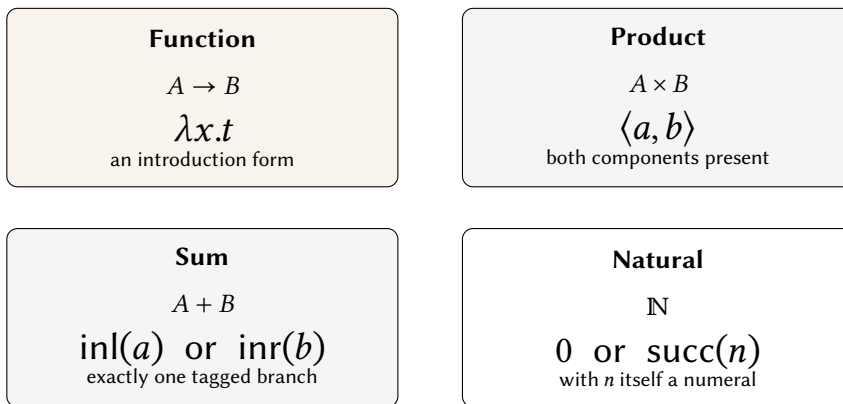


Figure 28: **Plate 28. Equality budget.** A language designer chooses how much equality the checker recognizes without explicit evidence. Moving the boundary can trade proof burden against conversion complexity, but the picture is not a claim of a strict zero-sum law: algorithmic advances and restricted principles can change the feasible frontier.

A type constrains the final shape of a closed normal term



Canonical-forms lemmas turn typing information into structural information about values.

Figure 29: **Plate 29. Canonical forms.** In the closed pure fragment taught here, a normal inhabitant's type sharply restricts its possible introduction form. This gallery is type-directed, not merely syntactic. Richer calculi can have additional canonical forms, so the lemma must always be stated for a particular language.

The normalization boundary: a loop that simple types refuse to admit

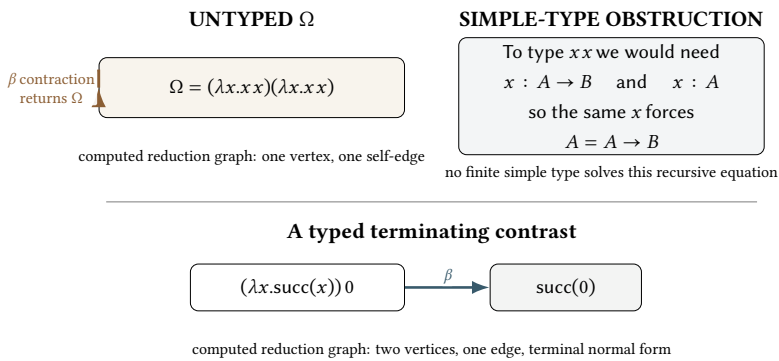


Figure 30: **Plate 30. Divergence boundary.** Contracting the root beta-redex of Ω reproduces exactly the same syntax, yielding a genuine self-loop in the reduction graph. In the ordinary simply typed lambda calculus, typing the subterm xx would force a recursive type equation $A = A \rightarrow B$, which finite simple types do not admit. Recursive or richer type systems change this particular argument; the plate illustrates one mechanism behind strong normalization of the simple calculus, not a universal prohibition on self-reference.

Type safety is two invariants working together

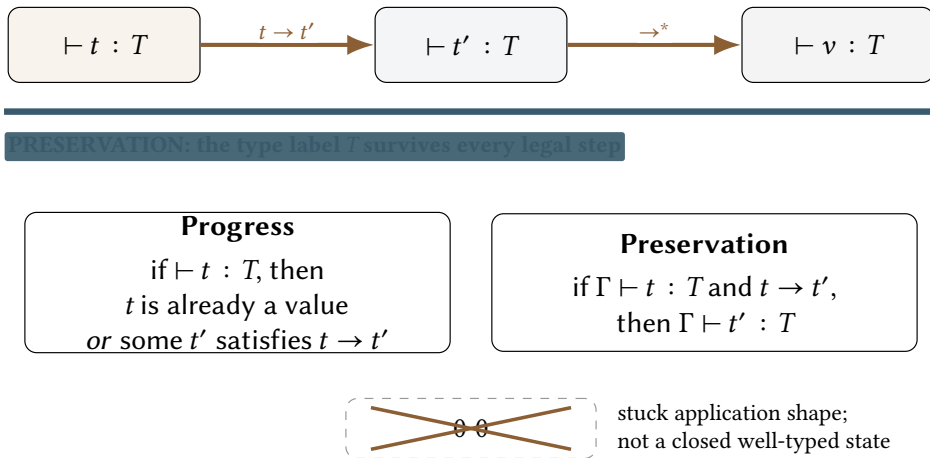


Figure 31: **Plate 31. Progress and preservation.** Progress excludes closed well-typed non-values with no legal move; preservation keeps the type invariant along every reduction step. Together they rule out stuck states for this pure core. They do not imply termination, correctness with respect to user intent, security, or resource bounds.

Curry–Howard lives in the rules, not in a slogan

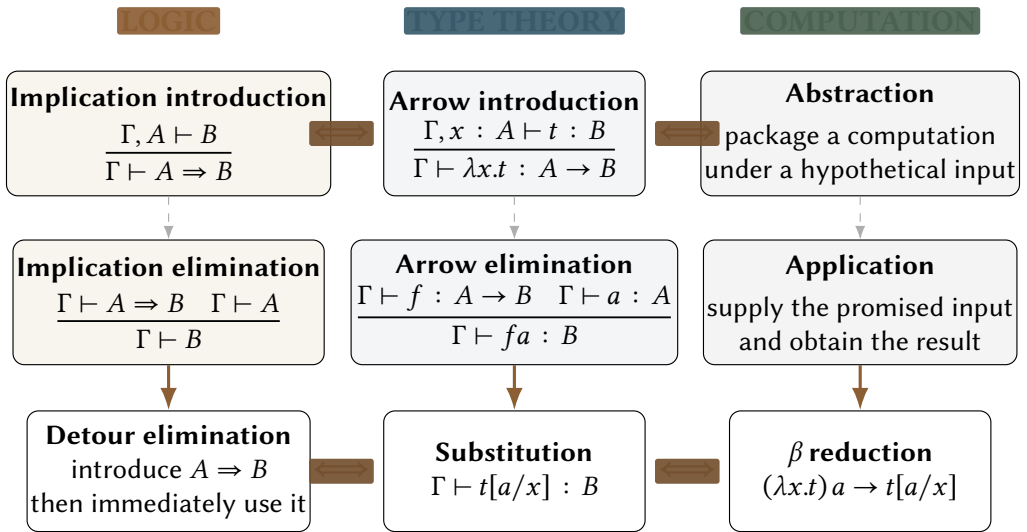


Figure 32: **Plate 32. Curry–Howard at rule level.** Implication introduction, arrow introduction, and lambda abstraction are three readings of the same structural move; implication elimination, arrow elimination, and application align likewise. The final row is the decisive dynamic correspondence: proof detour removal is substitution, and on terms that substitution is exposed by beta reduction. The plate concerns the constructive fragment developed here; richer logics and effects require additional structure.

A type can be treated as a construction search problem

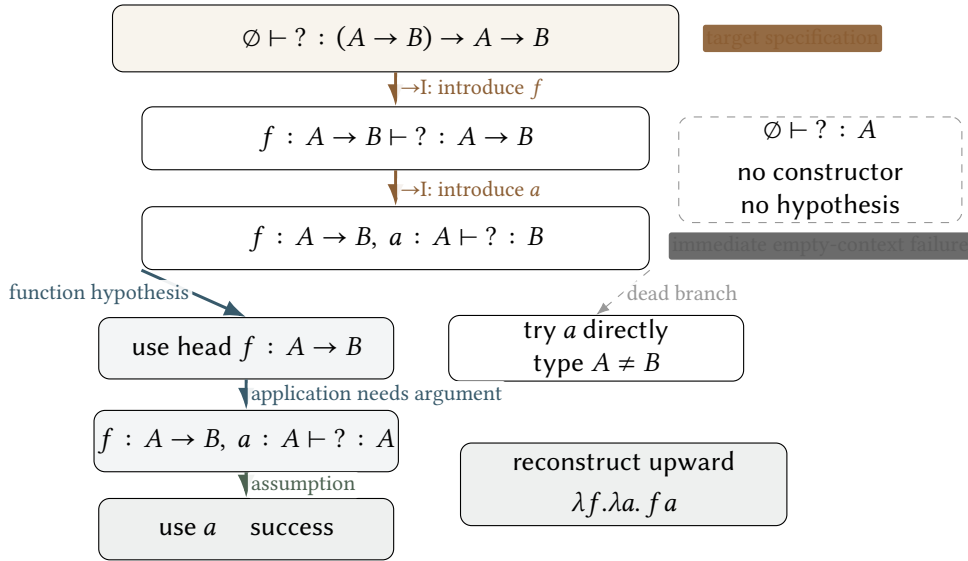


Figure 33: **Plate 33. Inhabitation as bounded search.** Backward proof search decomposes the target type into obligations, consults the context as an inventory of usable hypotheses, and reconstructs a term from a successful branch. The shipped search program finds $\lambda f. \lambda a. f a$ for the displayed target and rechecks the result with TinyTT. Failure of a bounded search is not, in general, a proof of uninhabitability; the atomic empty-context example is included because its failure is immediate from the fragment’s constructors.

Bidirectional typing routes information instead of guessing globally

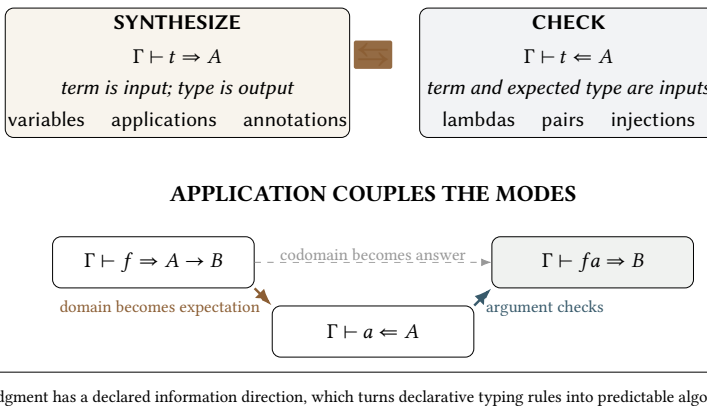


Figure 34: **Plate 34. Bidirectional typing.** Synthesis pushes information from syntax toward a type; checking pushes an expected type toward syntax. Application couples the modes: the function synthesizes an arrow, its domain becomes the expected type for the argument, and its codomain becomes the synthesized result. Bidirectionality does not change the declarative meaning of typing; it orients the rules into a predictable algorithmic discipline.

Make search powerful; keep acceptance small

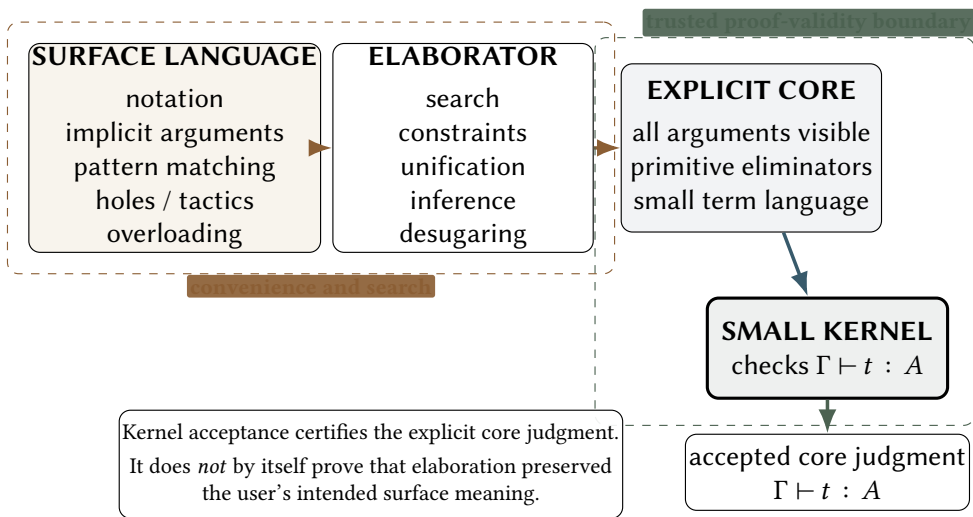


Figure 35: **Plate 35. Elaborator and trusted kernel.** A large elaborator may perform sophisticated, heuristic work while a smaller kernel verifies the explicit artifact it produces. This localizes trust for the formal typing claim: search need not itself be trusted if its output is independently checked. The boundary is not magical, however; faithful elaboration from surface syntax to core syntax is a separate semantic obligation.

A useful type error preserves the failed derivation path

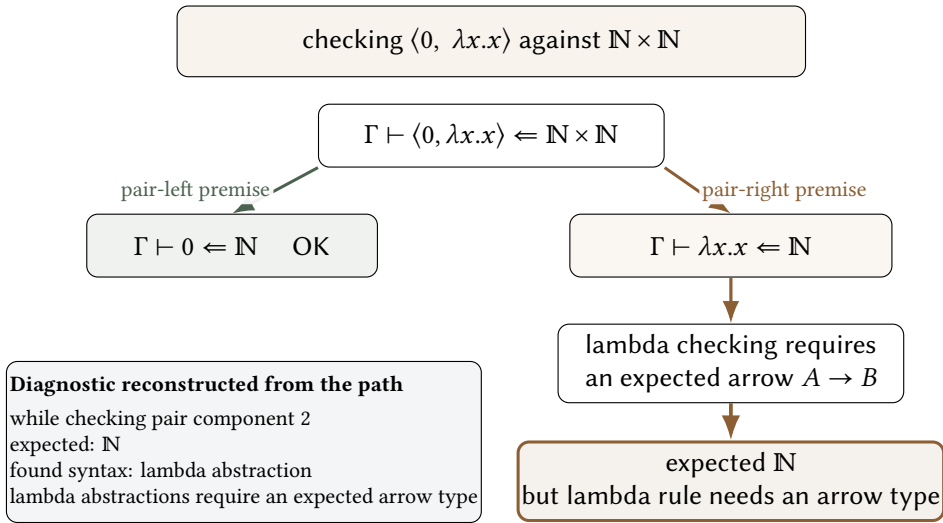


Figure 36: **Plate 36. Error as failed derivation.** The useful diagnostic is not merely the final inequality “expected $\mathbb{N}$, got something else.” It records which typing premise was being established, where that premise came from, and which rule became inapplicable. A derivation-oriented checker can therefore turn formal structure directly into causal user-facing diagnostics.

After the machinery is earned, a judgment can be read as communication

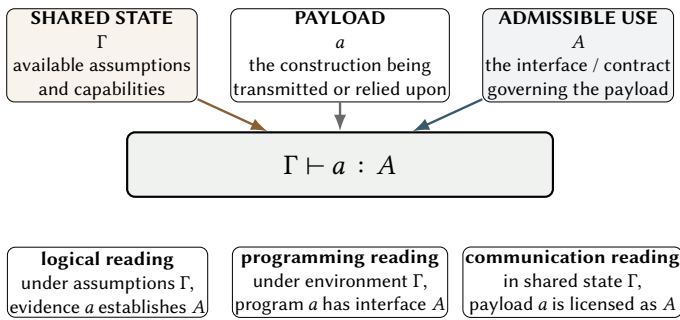
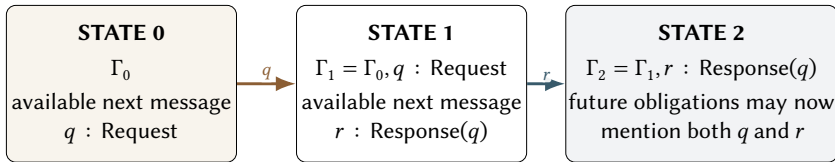


Figure 37: **Plate 37. Judgment as communication.** The communicative reading becomes precise only after contexts, substitution, constructors, eliminators, normalization, and checking have been developed. Here Γ is shared informational state, a is the construction under consideration, and A governs its admissible use. This is a reading of typing relative to information, not yet a full temporal protocol semantics; session structure arrives later in the series.

Communication can change the grammar of what may come next



The key change is not merely “more data.”
The newly available information may occur inside the type of a later legal message.

A typing context records assumptions; a protocol semantics adds temporal transition rules.
The picture combines them deliberately, but they are not identical notions.

Figure 38: **Plate 38. Context as protocol state.** After a request q is received, a later response contract may depend on that particular request, schematically $\text{Response}(q)$. Context extension therefore suggests an operational reading in which communication changes the space of legal continuations. The analogy has a boundary: ordinary typing contexts are not clocks, and context extension alone does not impose message order, liveness, duality, or any other temporal protocol law.

A type theory is a collection of licensed world-building moves

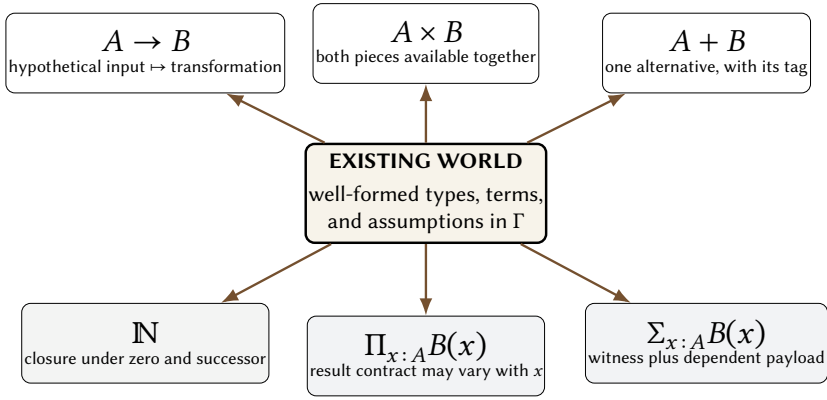


Figure 39: **Plate 39. Collected comprehension principles.** Each type former enlarges the language of admissible constructions in a characteristic way: arrows internalize hypothetical transformation, products coexistence, sums alternatives, naturals generated closure, and dependent products and sums information-indexed construction. The picture is a rule atlas, not an ontology of literal spatial worlds. The Π and Σ gates mark the threshold of Volume II rather than constructors already implemented by TinyTT.

Volume I forms a causal spine, then closes a verification loop

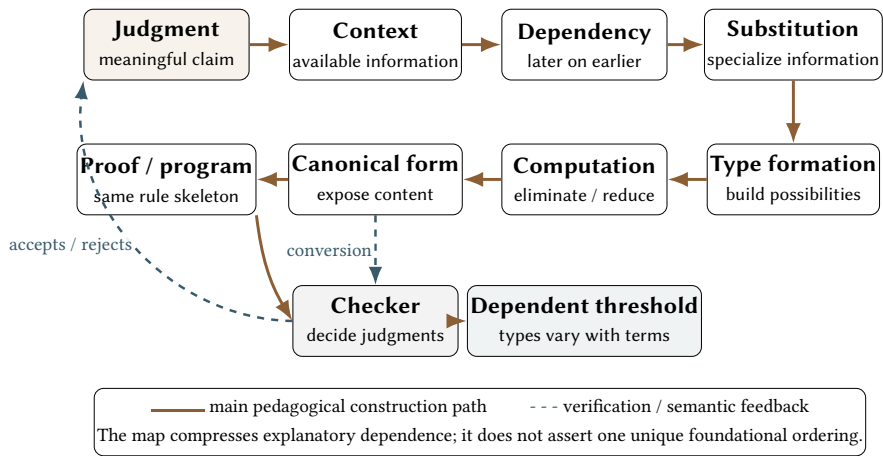


Figure 40: **Plate 40. Volume I synthesis.** The warm spine summarizes the book’s construction order: judgments become contextual, dependency becomes operational through substitution, type formation enables computation, normalization exposes canonical content, Curry–Howard re-reads the same rules, and an algorithmic checker returns to the original judgment before simple types reach their dependent boundary. Dashed arrows record feedback used in verification. The topology is pedagogical rather than a claim of unique foundational precedence.

Three different problems demand the same missing grammatical move

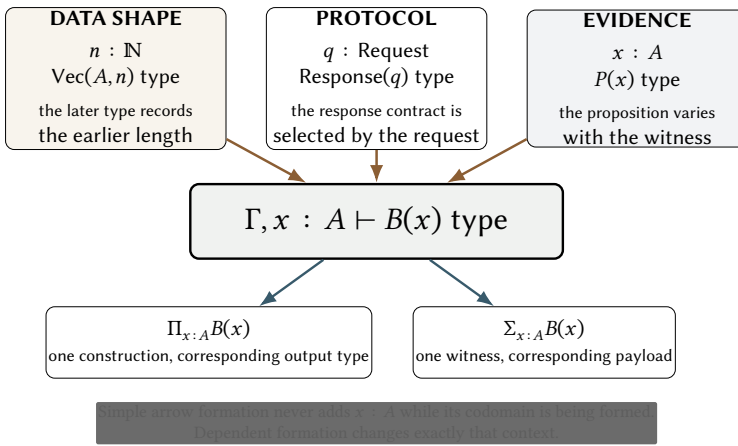


Figure 41: **Plate 41. Threshold to Volume II.** Length-indexed data, request-indexed responses, and witness-indexed propositions are structurally the same demand: after obtaining $x : A$, the language must be able to form a type $B(x)$ that mentions it. Dependent products and sums internalize two canonical ways of using such a family. The examples motivate the common judgment; they do not claim that vectors, protocols, and propositions have identical semantics.

The collection is one hypothesis examined from ten directions

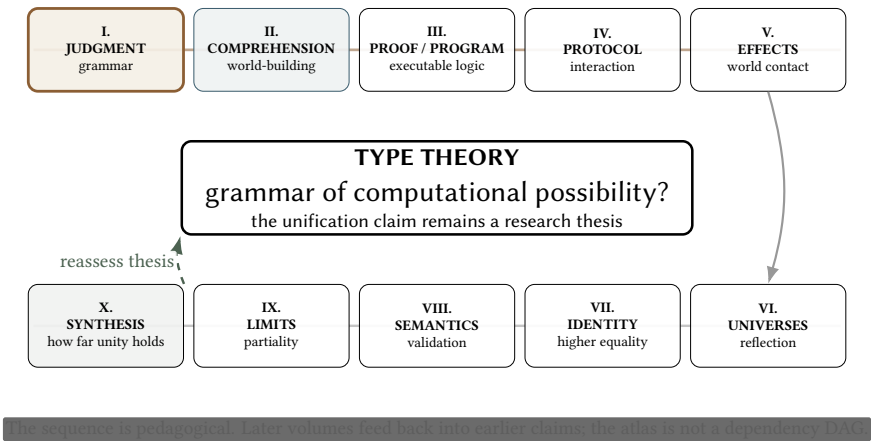


Figure 42: **Plate 42. Series atlas.** Volume I establishes the judgmental grammar and hands directly to comprehension in Volume II. The remaining monographs test the unification thesis against proof, interaction, effects, reflection, identity, semantics, and the limits of formal construction before attempting a synthesis. This is an editorial research map, not a theorem that the subjects form a strict hierarchy or a single linear prerequisite chain.